



Marine Mechatronics Control of Underwater Robots

Practical work: Implementation of an autonomous behavior on the BlueROV

Master MIR

Submitted By:

Student Name 1: Mahmoud Aboelrayat

ID: 22405136

Student Name 2: Mohamed Magdy Atta

ID: 22405169

Student Name 3: Shahid Ahamed Hasib

ID: 22405158

Student Name 4: Mohamed Alsisi

ID: 22405161

Submitted To

Prof. Cédric Anthierens

Prof. Vincent HUGEL

Universite de Toulon

2024/2025

May 28, 2025

Contents

1	Introduction	2
2	Proportional Controller for the Depth	3
3	Proportional Controller for the Yaw Rotation	3
4	Autonomous Behavior	4
5	Results	6
5.1	Surface Movement	6
5.2	Underwater Movement	7
6	Appendix	9

1 Introduction

Autonomous underwater vehicles, such as the BlueROV, require robust control strategies for precise navigation, obstacle avoidance, and station-keeping. This project aims to develop autonomous behavior for the BlueROV, focusing on maintaining a fixed heading while moving forward, detecting obstacles, and maneuvering around them to continue navigation. In addition, it focuses on moving the robot sideways, perpendicularly to a wall at a fixed distance from a wall.

The primary control components implemented in this study include a trajectory-following P controller for depth regulation, a P controller for heading and yaw control, and a Ping Sonar Echosounder for obstacle detection. Sensors utilized in the project include an Inertial Measurement Unit (IMU) for yaw angle measurements and a frontal echosounder pinger for detecting forward-facing obstacles. The experimental environment comprises a tank with static obstacles such as vertical wooden boards, tank walls, and other BlueROVs.

This practical work integrates theoretical concepts in marine mechatronics with hands-on experiments to develop autonomous navigation capabilities in underwater robots.

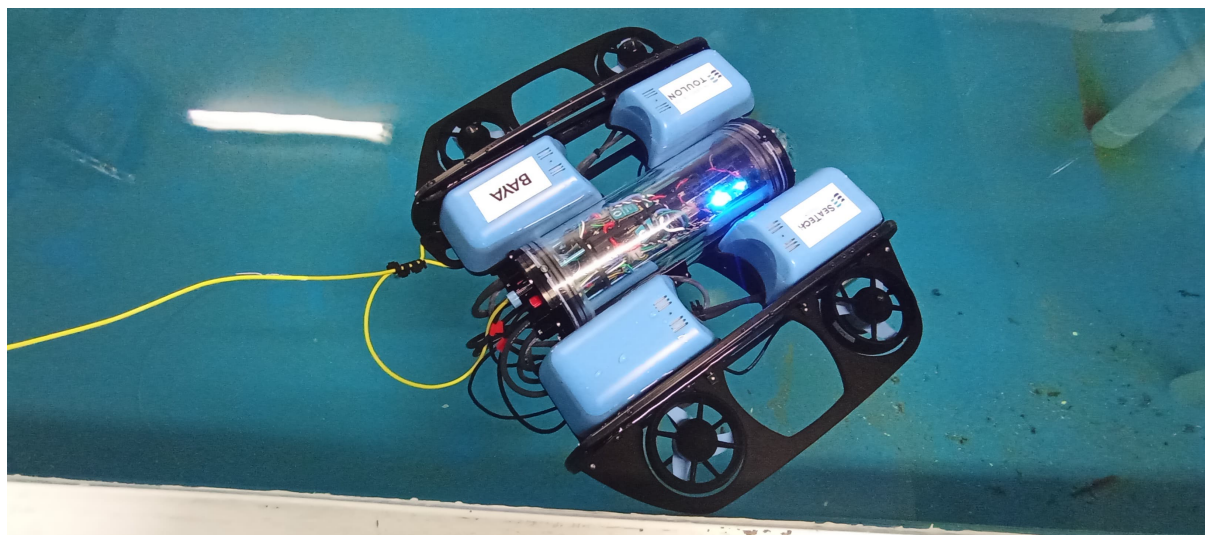


Figure 1: Blue ROV

2 Proportional Controller for the Depth

During our practical tests, we used proportional depth control to keep the robot at a constant depth during performing the autonomous mission:

$$\tau_z = K_p \tilde{z} + \text{floatability} \quad (1)$$

Where $\tilde{z} = (z_{des} - z)$ is the error along depth z , and K_p is the proportional gain of the P controller.

To test the P controller, we implemented it along with the polynomial trajectory. The following is the python implementation:

```
1 depth = data.data
2 error = self.z_desired - depth
3 self.step += 1
4 z, z_dot = self.cubic_trajectory(self.step / rate)
5 depth_trajectory = Float64()
6 depth_trajectory.data = z
7 self.pub_depth_trajectory.publish(depth_trajectory)
8 error = z - depth
9 force = self.kp * error - self.floatability
10 pwm = self.force_pwm(force/4)
11 # update Correction_depth
12 Correction_depth = pwm
13 self.Correction_depth = int(Correction_depth)
```

Listing 1: Python Code for polynomial trajectory with the depth P controller and floatability compensation

3 Proportional Controller for the Yaw Rotation

Yaw control is crucial for maintaining the orientation of an ROV. In this section, we implement both a Proportional (P) and a Proportional-Integral (PI) controller to regulate the yaw angle, ensuring stable heading control under disturbances.

Yaw control requires real-time feedback from the onboard IMU sensor. The yaw angle ψ is extracted from the sensor's quaternion representation and converted to roll, pitch, and yaw angles. This feedback is used to implement P control approach.

The problem involves implementing a proportional controller to manage the yaw (ψ) of the BlueROV. The goal is to control the yaw by adjusting the torque generated by horizontal thrusters. The controller in yaw implies creating a torque that is proportional to the yaw error.

The torque Γ_z created by the proportional controller is given by:

$$\Gamma_z = K_p \times \epsilon = K_p \times (\psi_{des} - \psi)$$

where ψ_{des} is the desired yaw, ψ is the ROV's measured yaw, and K_p is the proportional gain, which needs to be tuned.

Since yaw angles are cyclic between $-\pi$ and π , the error is normalized to ensure the ROV always takes the shortest turn:

$$e_{yaw} = (e_{yaw} + \pi) \mod 2\pi - \pi \quad (2)$$

Implementation in Python:

```
def normalize_angle(self, angle):
    # Ensure angle remains within [-180, 180]
    return (angle + np.pi) % (2 * np.pi) - np.pi
error_angle = self.yaw_desired + self.angle_yaw_a0 - angle_yaw
error_angle = self.normalize_angle(error_angle)
force = kp_yaw * error_angle
msg_d = Float64()
msg_d.data = float(self.yaw_desired)
self.pub_yaw_trajectory.publish(msg_d)
```

4 Autonomous Behavior

The autonomous navigation system is implemented as a state machine to achieve obstacle avoidance, free path exploration, and sideways motion. This approach enables the robot to navigate through unknown environments by combining forward motion, sideways motion, obstacle detection, and systematic exploration of alternative paths. The autonomous behavior is structured as a finite state machine with three primary operational states, as illustrated in Figure 2. The states are explained below:

- **State 1 : Free:** The robot maintains forward motion with a fixed heading using a surge motion value of 1520. This represents the default exploratory behavior where the robot advances through the environment until an obstacle or wall is detected.
- **State 2 : Hold:** When an obstacle or wall is detected, the robot transitions to a holding state. During this phase, the system calculates the error between current pinger distance and desired distance, implements a PI controller with error summation, and computes the control force to apply to the surge motion. The robot remains in this state until distance measurements stabilize within a small interval. This is achieved using the `distance_stable` function, which monitors recent pinger distance readings and checks if they remain within a specified range. The function is defined as follows:

```
1 def distance_stable(self, window_size=10, threshold=2.0, tolerance=0.5):
2     self.pinger_readings.append(self.pinger_distance)
3     if len(self.pinger_readings) < window_size:
4         return False
5     elif len(self.pinger_readings) > window_size:
6         self.pinger_readings.pop(0)
7     return max(self.pinger_readings) - min(self.pinger_readings) >= threshold
```

The function maintains a rolling window (specified by `window_size`) of the most recent pinger distance measurements stored in `self.pinger_readings`. Each time the function is called, the latest distance reading (`self.pinger_distance`) is appended to this list. If there are not yet enough readings to fill the window, the function returns `False`, indicating that stability cannot yet be determined. When the list exceeds the window size, the oldest reading is removed to keep the window

size fixed. Finally, the function checks whether the difference between the maximum and minimum values in the window is at least the specified **threshold**. If this condition is met, it returns **True**, indicating that the distance readings are stable.

- **State 3 : Mission Specific Task:** Upon stabilization, the robot select a third state to transition to it depending on the mission:
 - **Explore:** The robot performs a rotational search, slowly rotating on the spot while monitoring distance measurements from the echosounder pinger to find a free path.
 - **SideWays:** The robot executes lateral motion (sway) and monitor the pinger distance to maintain a fixed distance from a wall, enabling wall-following behavior.

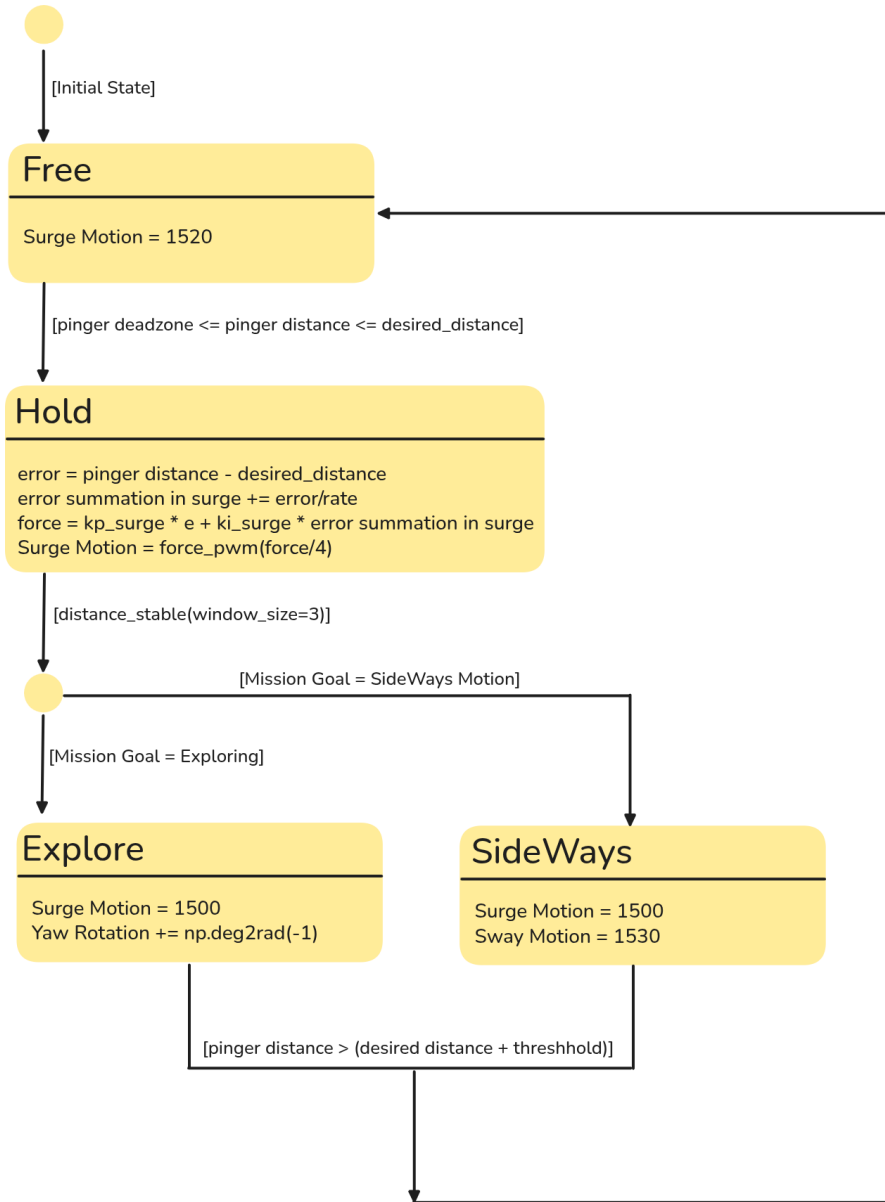


Figure 2: Autonomous Behavior State Machine

5 Results

5.1 Surface Movement

During the practical experiments of our autonomous state machine, we started the free movement on the surface to detect walls or obstacles then stop 1.3m approximately away from the detections. This behavior is caused by the pinger readings being not stable and low frequent (only one reading per second), which forced us to reduce the PI controller duration and the robot stopping distance became 1.3 m instead of 1 m. In addition, A constant oscillation in yaw rotation was observed as the BlueROV tries to maintain the correct heading angle. The following plots show the described behaviors. We also noticed that the pinger readings large increase to above 3 meter was because of a gap in one the the objects placed in the pool.

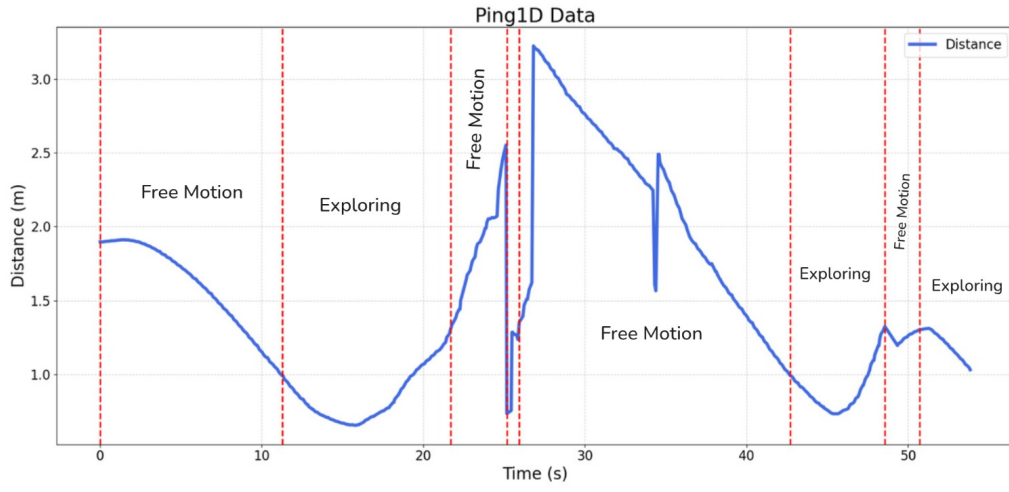


Figure 3: Distance from the obstacles or walls

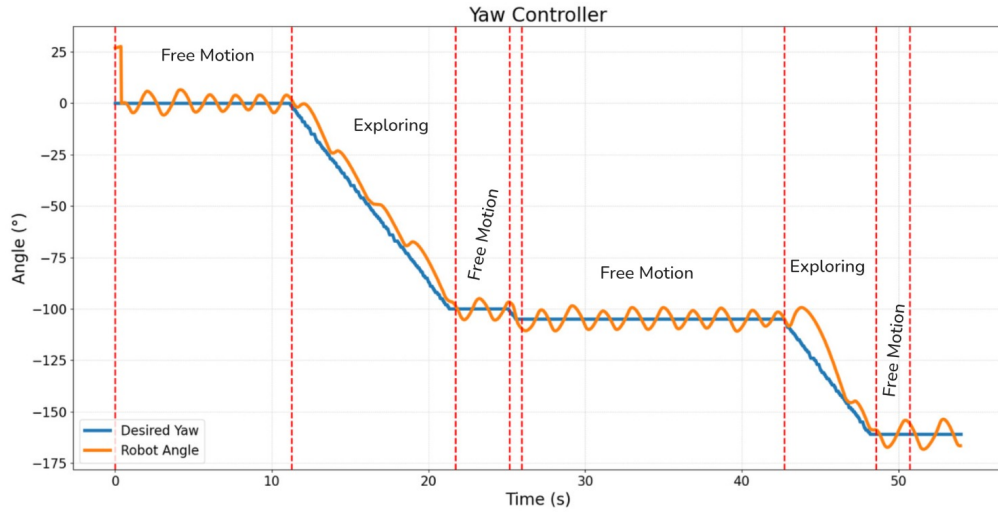


Figure 4: Yaw Readings during movement

The following camera frames show the moments the robot free motion was altered to avoid collision.



Figure 5: Obstacles/Walls during surface movements

5.2 Underwater Movement

After the surface trials, we moved to the underwater ones. We set the depth to 0.5 m underwater. The robot moved to the desired depth using a P controller, then kept a constant depth and performed the exploring mission. During the underwater mission, there were a lot of objects other than the walls in the pool, thus the robot changed its direction a lot to avoid collisions. A constant oscillation in yaw rotation was observed underwater as well but the oscillations amplitude was smaller than the surface mission. In addition, due to the angle being normalized, a change from negative angle at 180 degrees to positive angle at 180 happens, in order to keep the angle in the permissible range.

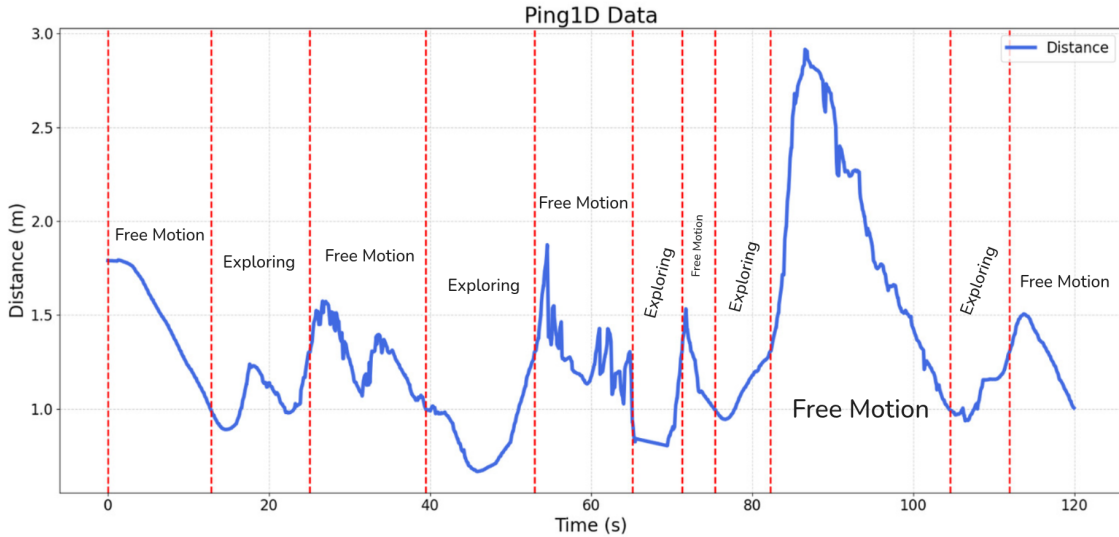


Figure 6: Distance from the obstacles or walls

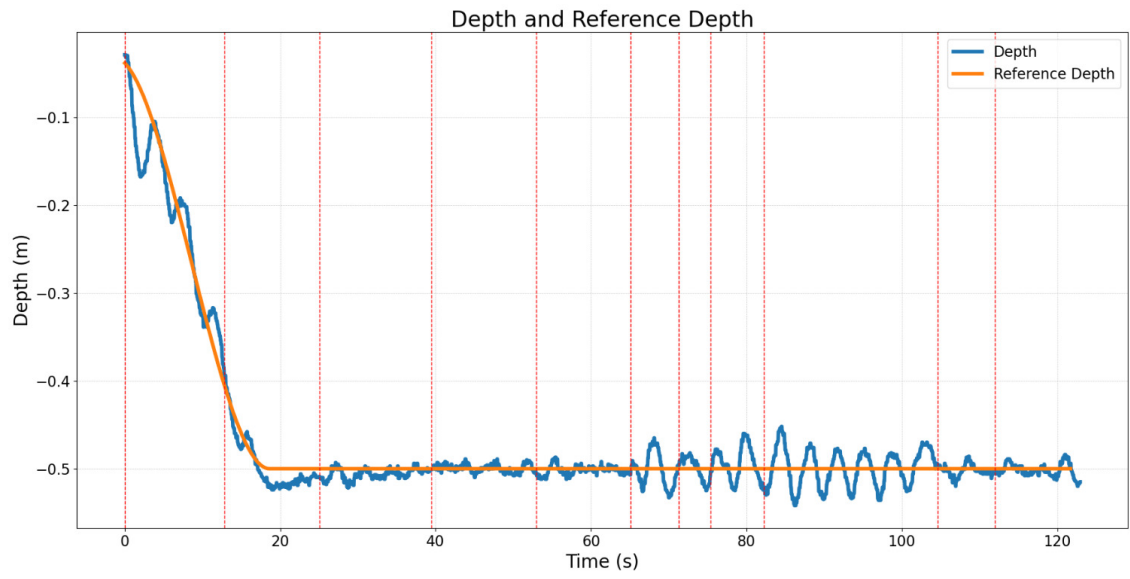


Figure 8: Depth Data

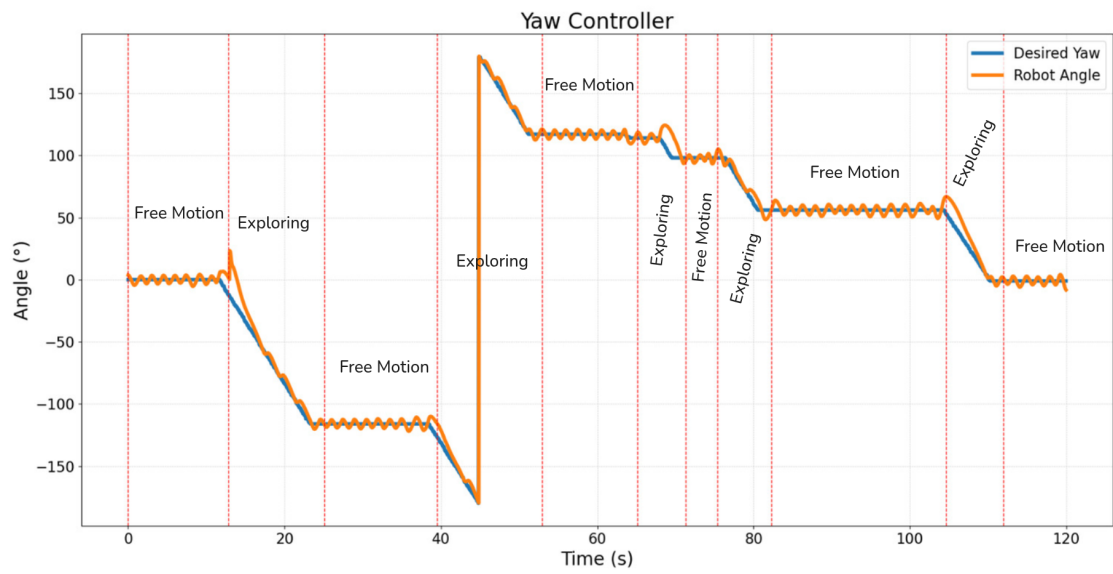


Figure 7: Yaw Readings during movement

The following camera frames show the moments the robot free motion was altered to avoid collision.



Figure 9: Obstacles/Walls during underwater movements

6 Appendix

We uploaded all the code we developed for the BlueROV practical to a github repository with video demonstrations of the BlueROV performance:

[Github Link](#)